

Coding & Solution

Date	02 July 2026
Team ID	xxxxxx
Project Name	AI Powered Debt Relief & Financial Recovery Platform
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Sabiha12-code/AI-Powered-Debt-Relief-and-Financial-Recovery-Platform.git
Programming Language(s)	Python, JavaScript, HTML, CSS
Framework(s) Used	FastAPI (Backend), React.js with Vite (Frontend), SQLAlchemy (ORM)

Key Features Implemented	• User Authentication: Secure user registration and login using JWT authentication. • Loan Management: Add, view, and manage loan details. • Financial Analysis: Analyze financial data and provide AI-powered debt relief recommendations. • AI Negotiation Letters: Generate debt negotiation letters using the Google Gemini API. • Financial Dashboard: Display financial summary, loan details, and recovery insights.
Pending / Incomplete Features	• Public deployment of the application. • Migration from SQLite to a production database. • Future enhancements to AI recommendations and user interface.
Setup / Run Instructions	Backend: Activate the virtual environment, install dependencies, configure .env, and run the FastAPI server. Frontend: Run npm install, npm run dev, and open http://localhost:5173.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments

The project uses a clean architecture with a FastAPI backend and a React/Vite frontend. It performs dynamic financial calculations using backend data instead of static values. The code follows a modular structure with clear and consistent naming conventions. Basic error handling is implemented to improve application stability and maintainability.